

Chapter 2: Text Chunking — Complete Code Guide (v2)

Exhaustive Mapping of Repository Scripts, .env References & Textbook Code Listings

Note on Filenames & .env.example References:

This v2 guide incorporates the 3 scripts explicitly referenced in the 02-DocChunking/.env.example file and printed in the textbook listings: 04-LangChain-ChunkingForCode.py, 04-LangChain-PlainChunkingForCode.py, and 05-LlamaIndex-SemanticChunking.py. Below is the complete mapping of all GitHub repo files and textbook code stamps.

1. Exhaustive Chapter 2 Code Inventory

Script / File Name	Origin / Status	Book Reference	Core Purpose & Mechanics
01-LangChain-CharacterTextSplitter.py	GitHub Repo	pp. 76–78	Fixed-character chunking using CharacterTextSplitter (chunk_size=100, chunk_overlap=10).
02-LangChain-RecursiveCharacterTextSplitter.py	GitHub Repo	pp. 79–81	Hierarchical recursive chunking using prioritized separators ['\\n\\n', '.', '!', '?', ' '].
03_LlamaIndex-ChunkSizeAffectsAccuracy.py	GitHub Repo	pp. 71–73, 79	Evaluates how Settings.chunk_size affects node generation, retrieval precision, and QA.
04-LangChain-ChunkingForCode.py	Book Listing / .env	pp. 81–82	Language-aware code chunking via RecursiveCharacterTextSplitter.from_language(Language.PYTHON).
04-LangChain-PlainChunkingForCode.py	Book Listing / .env	pp. 81–82	Plain character splitting on source code to demonstrate syntax disruption vs. code chunking.
05-LlamaIndex-SemanticChunking.py	Book Listing / .env	pp. 85–86	Semantic chunking using SemanticSplitterNodeParser with embedding similarity thresholds.
99-Tool-PDF-Splitting.py	Repo Utility	N/A	Utility using pypdf (PdfReader/PdfWriter) to extract page ranges (e.g. pp. 26-28) for test samples.
.env.example	Repo Config	N/A	Config template explicitly referencing OPENAI_API_KEY for 05-LlamaIndex-SemanticChunking.py.
requirements.txt	Repo Config	N/A	Package manifest listing langchain-text-splitters, llama-index-core, pypdf, etc.

2. Complete Code Stamps for Named Scripts & Textbook Listings

04-LangChain-ChunkingForCode.py (Language-Aware Code Splitting, pp. 81–82)

```
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

GAME_CODE = '''
class CombatSystem:
    def __init__(self):
        self.health = 100
        self.stamina = 100
    def update(self, delta_time):
        self._update_stats(delta_time)
        self._handle_combat()
    def _handle_combat(self):
        if self.stamina > 30:
            return 50 # Damage
'''

# Language-aware code splitter preserves class and function structure
python_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=100, chunk_overlap=0
)
python_docs = python_splitter.create_documents([GAME_CODE])
for i, doc in enumerate(python_docs, 1):
    print(f'--- Code Chunk {i} ---\n{doc.page_content}')
```

04-LangChain-PlainChunkingForCode.py (Plain Splitting Baseline, pp. 81–82)

```
from langchain_text_splitters import CharacterTextSplitter

# Plain character splitter cuts mid-function, destroying syntax context
plain_splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=0, separator='\n')
plain_docs = plain_splitter.create_documents([GAME_CODE])
for i, doc in enumerate(plain_docs, 1):
    print(f'--- Plain Chunk {i} ---\n{doc.page_content}')
```

05-LlamaIndex-SemanticChunking.py (Semantic Chunking, pp. 85–86)

```
from llama_index.core import SimpleDirectoryReader
from llama_index.core.node_parser import SemanticSplitterNodeParser, SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from dotenv import load_dotenv
load_dotenv()

# Initialize embedding model for semantic similarity calculation
embed_model = HuggingFaceEmbedding(model_name='BAAI/bge-small-zh')
documents = SimpleDirectoryReader(input_files=['data/black_myth_setting.txt']).load_data()

# Configure semantic splitter with 95th percentile breakpoint threshold
semantic_splitter = SemanticSplitterNodeParser(
    buffer_size=1,
    breakpoint_percentile_threshold=95,
    embed_model=embed_model
)
semantic_nodes = semantic_splitter.get_nodes_from_documents(documents)
print(f'Semantic Chunks Generated: {len(semantic_nodes)}')
```

Unstructured Document Structure Chunking ('by_title', pp. 83–84)

```
from langchain_unstructured import UnstructuredLoader

# Layout & structure-aware chunking closes chunks on title boundaries
loader = UnstructuredLoader(
    'data/black_myth_setting.txt',
    chunking_strategy='by_title',
    max_characters=1000,
    include_orig_elements=False
)
docs = loader.load()
print(f'Structure Chunks: {len(docs)}, First Chunk Length: {len(docs[0].page_content)}')
```

Advanced Indexing: Sentence Window & Parent-Child Chunking (pp. 87–90)

```
# 1. Sentence Window Node Parser (LlamaIndex, pp. 87–88)
from llama_index.core.node_parser import SentenceWindowNodeParser
window_parser = SentenceWindowNodeParser.from_defaults(
    window_size=3, window_metadata_key='window', original_text_metadata_key='original_text'
)
window_nodes = window_parser.get_nodes_from_documents(documents)

# 2. Parent-Child Document Retriever (LangChain, pp. 88–90)
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=50)
vectorstore = Chroma(collection_name='child_chunks', embedding_function=embed_model)
store = InMemoryStore()
retriever = ParentDocumentRetriever(
    vectorstore=vectorstore, docstore=store,
    child_splitter=child_splitter, parent_splitter=parent_splitter
)
retriever.add_documents(documents)
```

3. Chunking Strategy Selection Matrix

Strategy Name	Primary Use Case	Pros	Cons
Character / Fixed	Quick baselines, plain text	Fast, simple implementation	Risks cutting sentences/words mid-thought
Recursive Character	General prose, markdown, docs	Preserves paragraph & sentence units	May still split complex table or code blocks
Code (Language)	Python, JS, C++, Go source code	Keeps class & function blocks intact	Requires language grammar parser
Unstructured (by_title)	PDFs, Word docs, HTML reports	Respects visual titles & headers	Higher parsing overhead
Semantic Chunking	Topic-heavy text, long articles	Ensures high semantic coherence	Slow; requires embedding model pass
Sentence Window	Granular detail lookup RAG	High retrieval precision + rich context	Increases vector store metadata overhead
Parent-Child	Complex multi-paragraph Q&A;	Decouples retrieval unit from synthesis	Requires dual-store architecture